

ECE 382N-Sec (FA26):

L12: Physical-ish Attacks

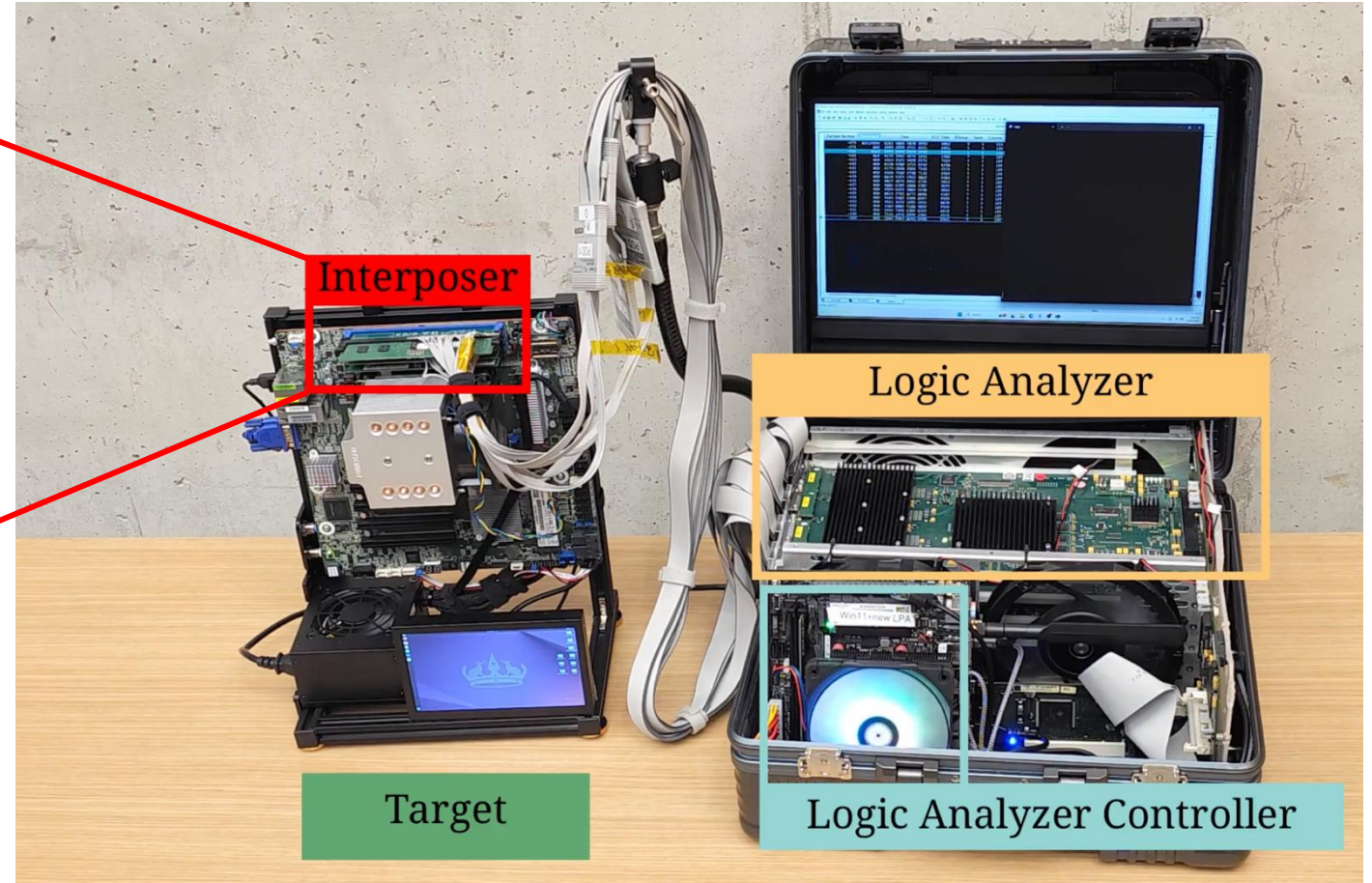
Neil Zhao

neil.zhao@utexas.edu

Physical Attacks – WireTap¹ and TEE.Fail²



Passive observations



¹Seto et al., "WireTap: Breaking Server SGX via DRAM Bus Interposition," CCS '25

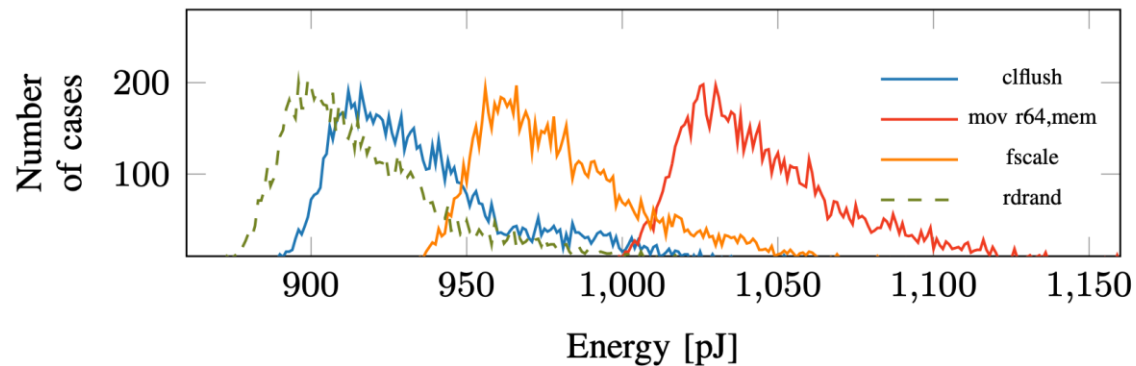
²Chuang et al., "TEE.fail: Breaking Trusted Execution Environments via DDR5 Memory Bus Interposition," SP '26

Physical Attacks – Simple Power Analysis

Instruction	Xeon Silver 4214	i7-6700K	i7-8650U
nop	0.1795 nJ	0.1189 nJ	0.0843 nJ
inc r64	0.1795 nJ	0.1208 nJ	0.0858 nJ
xor r64, r64	0.1795 nJ	0.1209 nJ	0.0849 nJ
mov r64, mem	0.1868 nJ	0.1247 nJ	0.0840 nJ
imul r64, r64	0.1798 nJ	0.1169 nJ	0.0887 nJ
fscale	0.1867 nJ	0.1182 nJ	0.0877 nJ
rdrand r64	0.1797 nJ	0.1129 nJ	0.0982 nJ
rdtsc	0.1864 nJ	0.1189 nJ	0.0848 nJ
clflush mem	0.1865 nJ	0.1129 nJ	0.1018 nJ
aesenc xmm, xmm	0.1794 nJ	0.1188 nJ	0.0946 nJ

Different instruction types have different energy consumptions

Secret-dependent control flow
⇒ Different power trace



Lipp et al., "PLATYPUS: Software-based Power Side-Channel Attacks on x86," SP '21

Side Channel Example: Leaky String Comparison

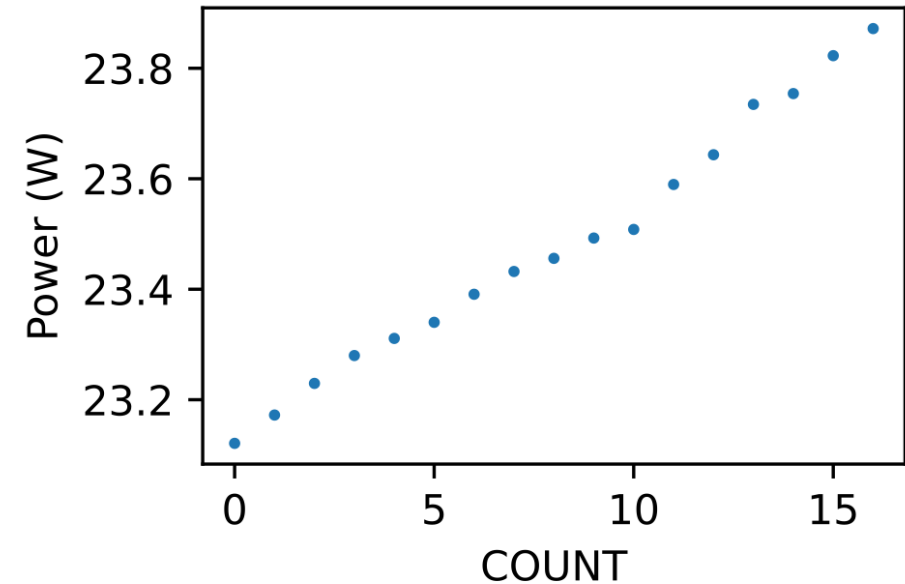
A naïve implementation that compares strings s1 and s2

```
bool str_equal(const char *s1, const char *s2) {  
    for (; *s1 && *s2; s1++, s2++) {  
        if (*s1 != *s2) return false; // Early return  
    }  
    return *s1 == *s2;  
}
```

Observe the comparison time through a power side channel

Data-Dependent Power Consumption Variation

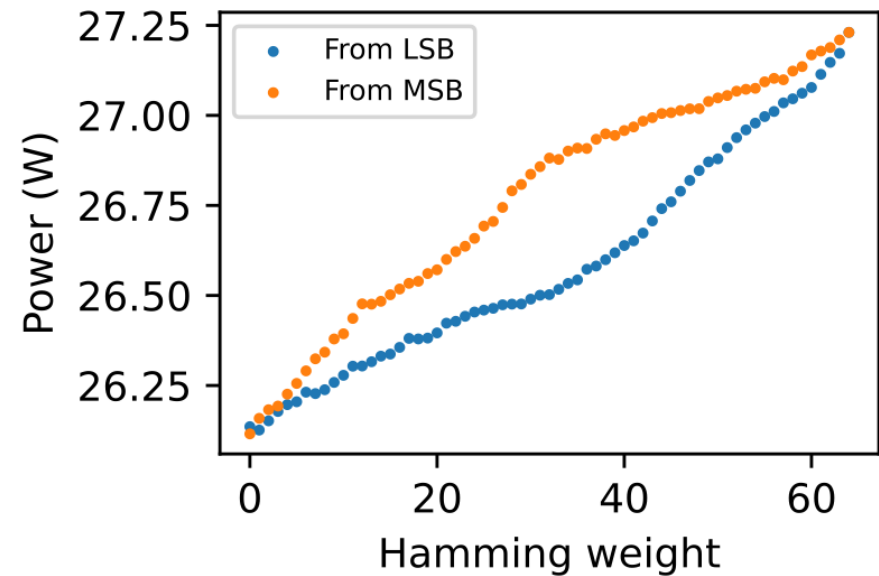
```
rax = COUNT
rbx = 0x0000FFFFFFFF0000
loop:
    shlx %rax,%rbx,%rcx    // rcx = rbx << rax
    shlx %rax,%rbx,%rdx    // rdx = rbx << rax
    shrx %rax,%rbx,%rsi    // rsi = rbx >> rax
    shrx %rax,%rbx,%rdi    // rdi = rbx >> rax
    shlx %rax,%rbx,%r8     // r8  = rbx << rax
    shlx %rax,%rbx,%r9     // r9  = rbx << rax
    shrx %rax,%rbx,%r10    // r10 = rbx >> rax
    shrx %rax,%rbx,%r11    // r11 = rbx >> rax
    jmp loop
```



Hamming Distance (HD) Effect

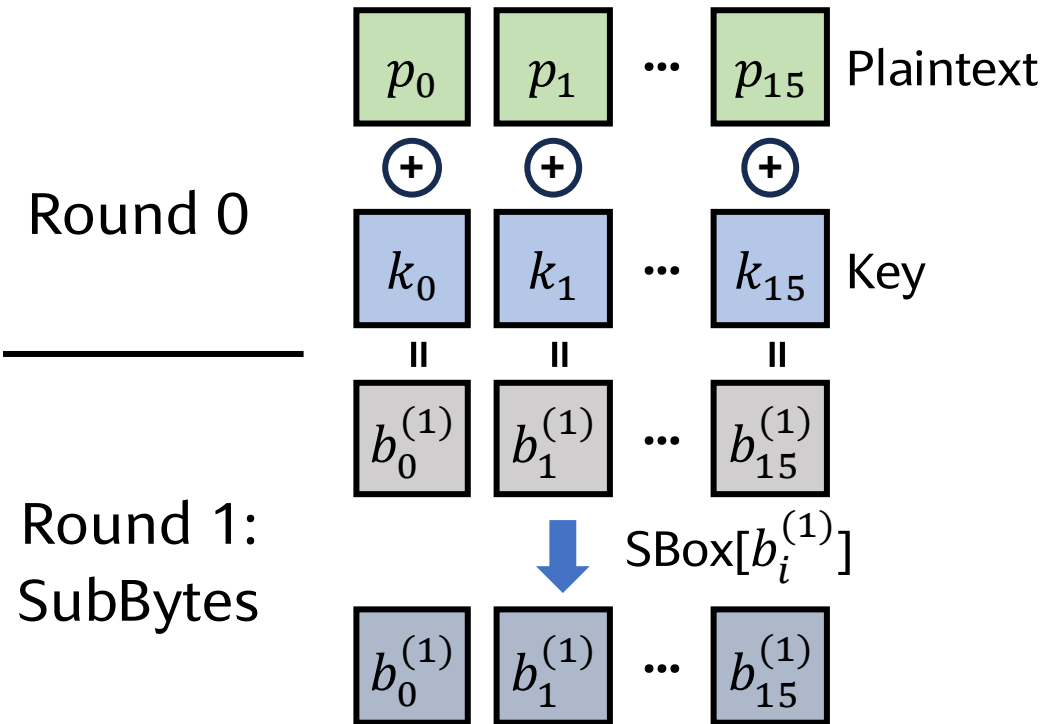
Data-Dependent Power Consumption Variation

```
rax = LEFT                LEFT=RIGHT
rcx = ... = r11 = RIGHT
loop:
    or %rax,%rcx          // rcx = rax | rcx
    or %rax,%rdx          // rdx = rax | rdx
    or %rax,%rsi          // rsi = rax | rsi
    or %rax,%rdi          // rdi = rax | rdi
    or %rax,%r8           // r8  = rax | r8
    or %rax,%r9           // r9  = rax | r9
    or %rax,%r10          // r10 = rax | r10
    or %rax,%r11          // r11 = rax | r11
    jmp loop
```



Hamming Weight (HW) Effect

Correlation Power Analysis Attack in AES



Leakage Model:

The power consumption of the SBox operation correlates with the **hamming weight** of the output byte

Correlation Power Analysis Attack in AES

		Key byte candidates		Output byte	HW
Plaintext byte: 0x80	⊕	0x00	SubBytes →	0xcd	5
		0x01		0x0c	2
		0x02		0x13	3
		...		...	...
		0xfe		0xf3	6
		0xff		0xd2	4

Measured Power: 5.65W

Correlation Power Analysis Attack in AES

		Key byte candidates			Output byte	HW
Plaintext byte: 0x40	⊕	0x00	SubBytes →		0x09	2
		0x01			0x83	3
		0x02			0x2c	3
		...			...	...
		0xfe			0xae	5
		0xff			0x08	1

Measured Power: 5.64W

Correlation Power Analysis Attack in AES

Different measurements using various inputs

Key Byte Candidates	HW	HW	HW	HW	HW	Abs(Correlation)
0x00	2	5	5	7	6	0.36
0x01	3	2	4	4	3	0.004
0x02	3	3	6	1	4	0.99
...	...	...	...	...	...	...
0xfe	4	5	3	3	6	0.07
0xff	3	1	2	4	1	0.58
Measured Power:	5.65W	5.64W	5.87W	5.42W	5.68W	



A blue arrow labeled "Corr" points from the "Measured Power" row to the "0x02" row, indicating a correlation between the power measurements and the key byte candidate 0x02.

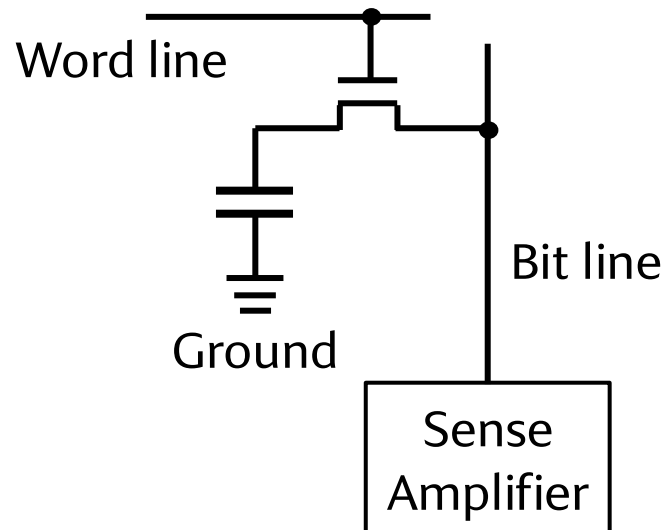
Correlation Power Analysis Attack in AES

After Power Trace Alignment

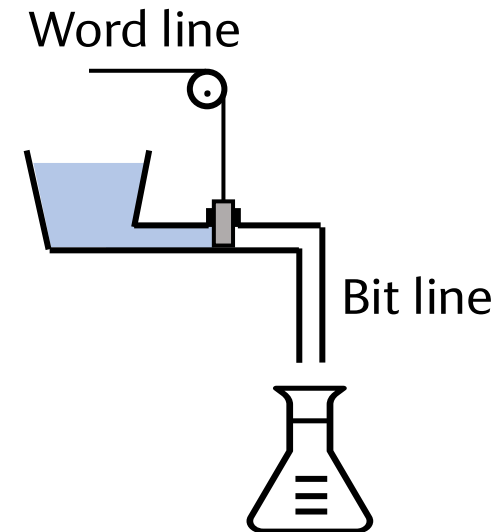
Power Trace #	T=0	T=1	T=2	T=3	T=4
1	5.31W	5.51W	5.65W	5.45W	5.90W
2	5.35W	5.42W	5.64W	5.57W	5.46W
3	5.42W	5.68W	5.87W	5.60W	5.85W
...	...	...	...	...	...
Candidate Byte	Abs(Correlation) at Each Timestamp				
0x00	0.32	0.36	0.26	0.16	0.12
0x01	0.12	0.00	0.12	0.21	0.13
0x02	0.23	0.98	0.05	0.32	0.24

DRAM Refresher: A Textbook DRAM Cell

A single DRAM cell
(one-transistor, one-capacitor)



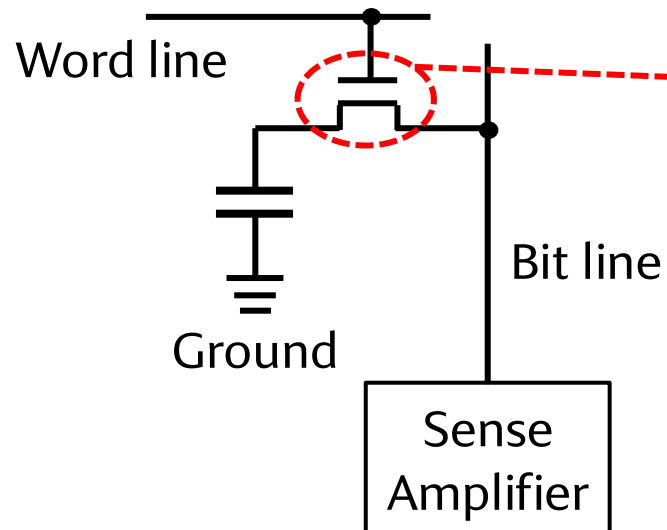
The water bucket analogy
(one-bucket, one-valve)



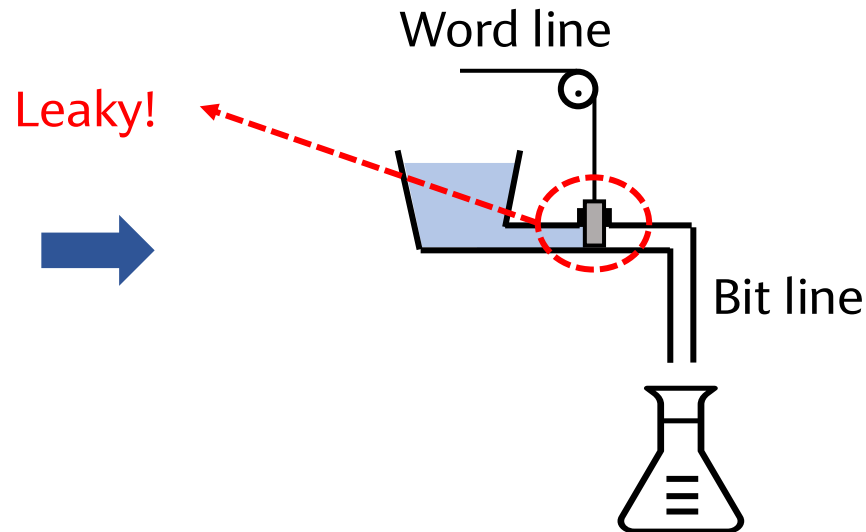
Storing 1 bit. Destructive reads

DRAM Refresher: A Textbook DRAM Cell

A single DRAM cell
(one-transistor, one-capacitor)



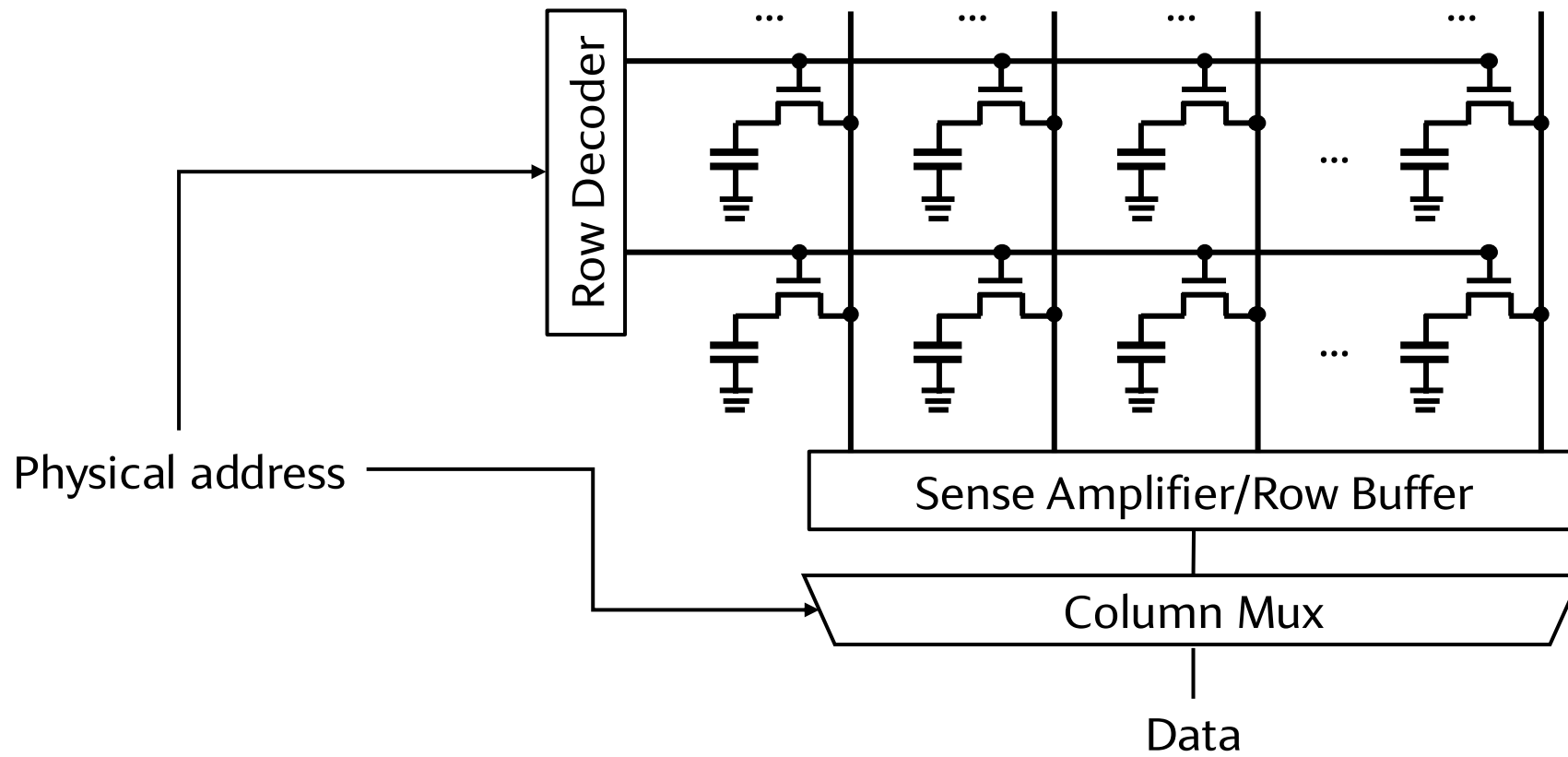
The water bucket analogy
(one-bucket, one-valve)



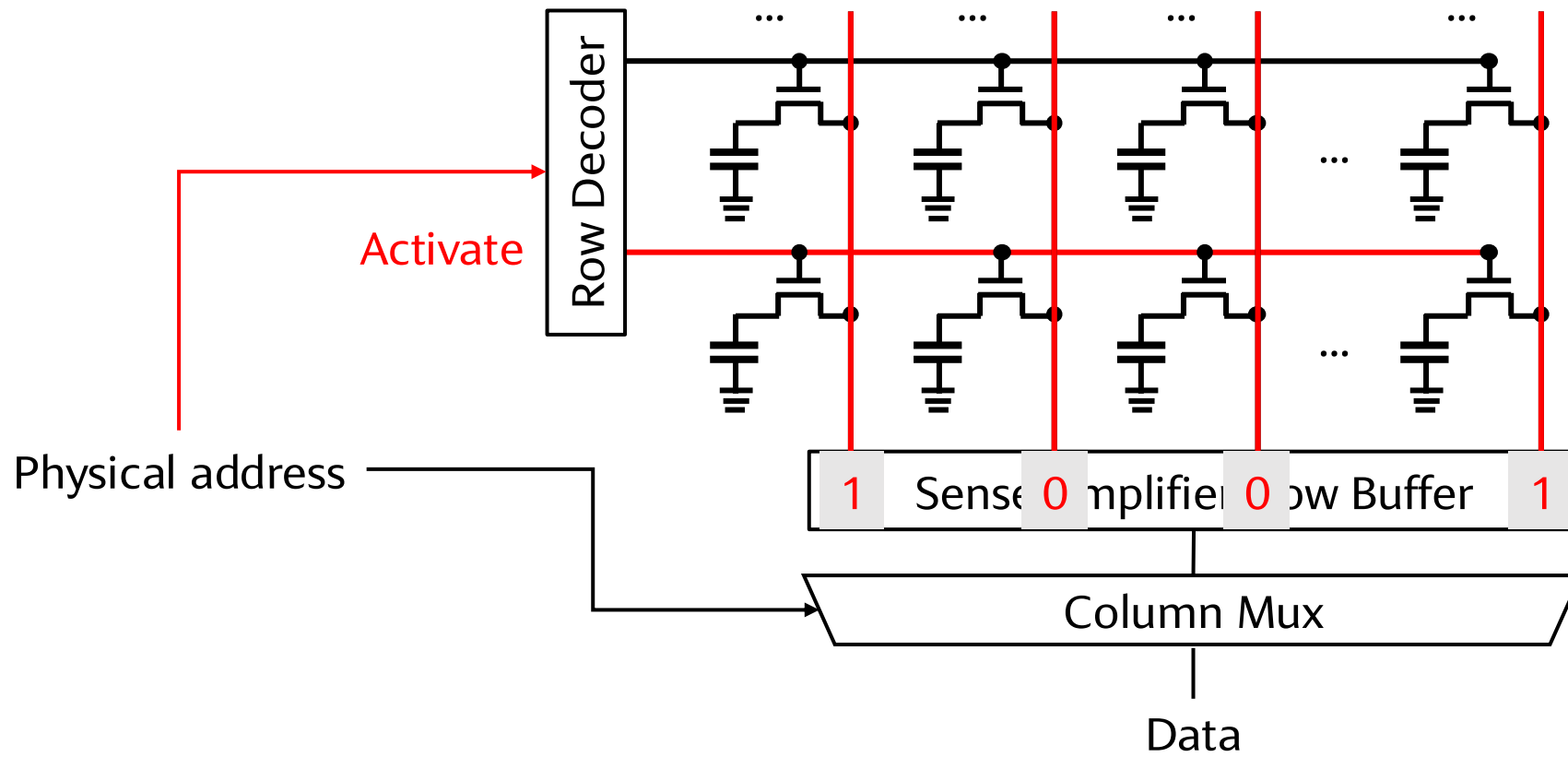
Solution*: Periodic refreshing (e.g., read it out and write it back every 64ms)

*Or spray it with liquid nitrogen!

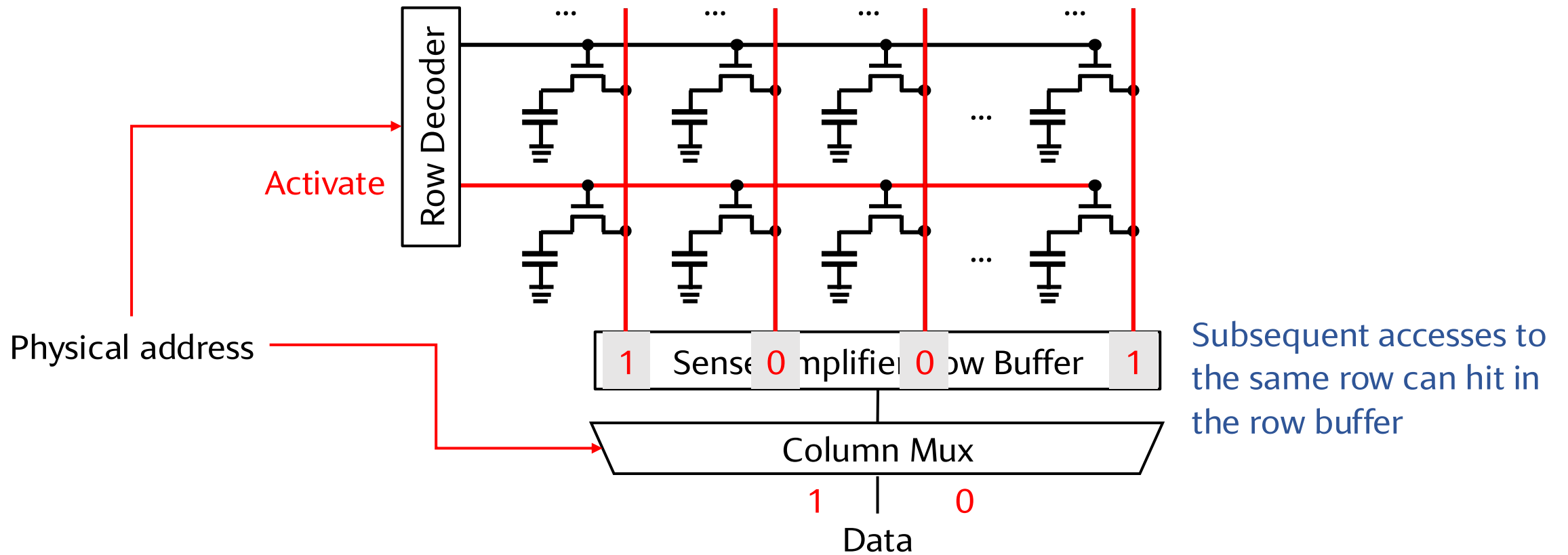
From a Single Cell to a DRAM Array



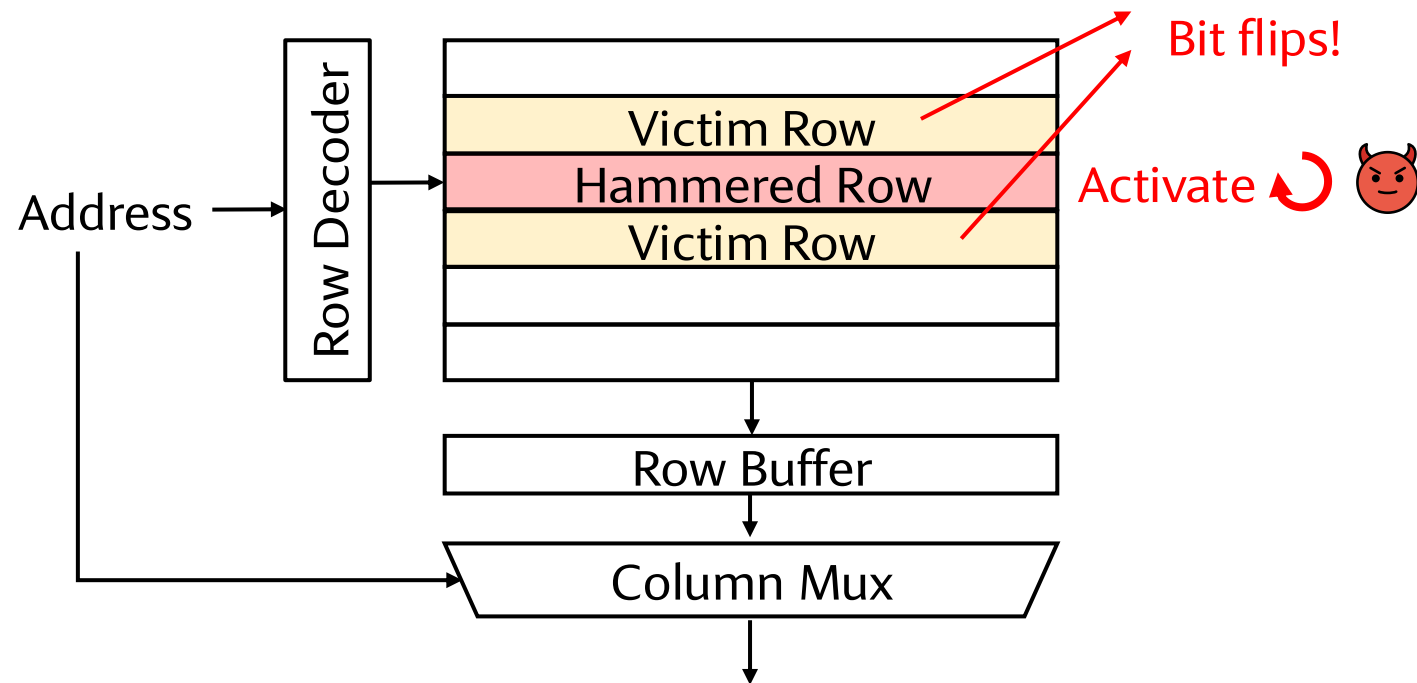
From a Single Cell to a DRAM Array



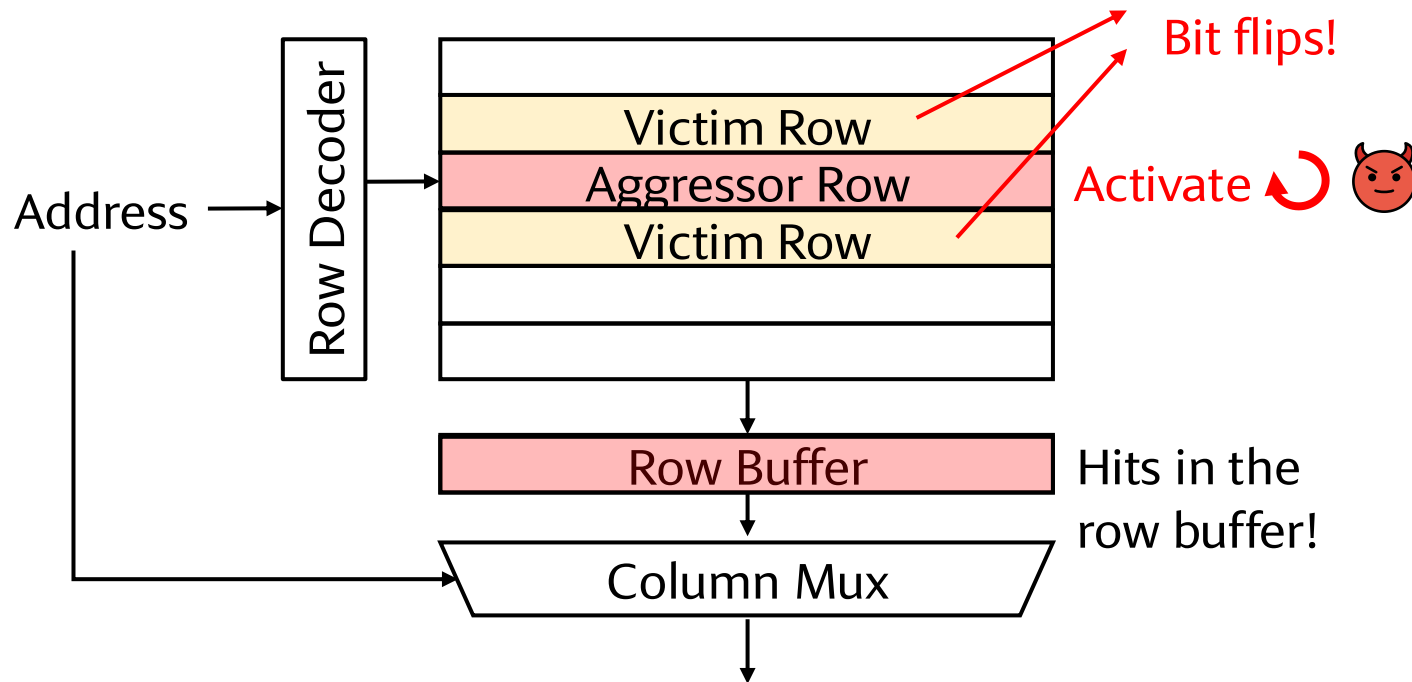
From a Single Cell to a DRAM Array



RowHammer

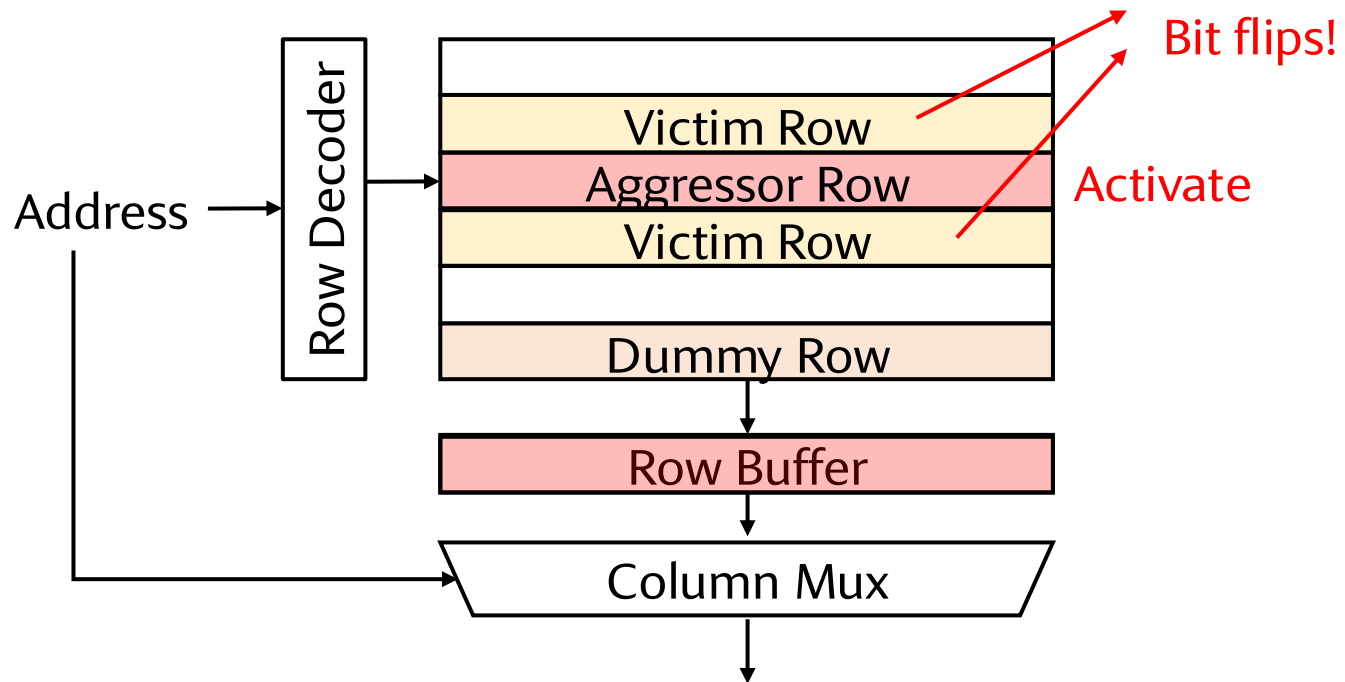


Hammering Sequence



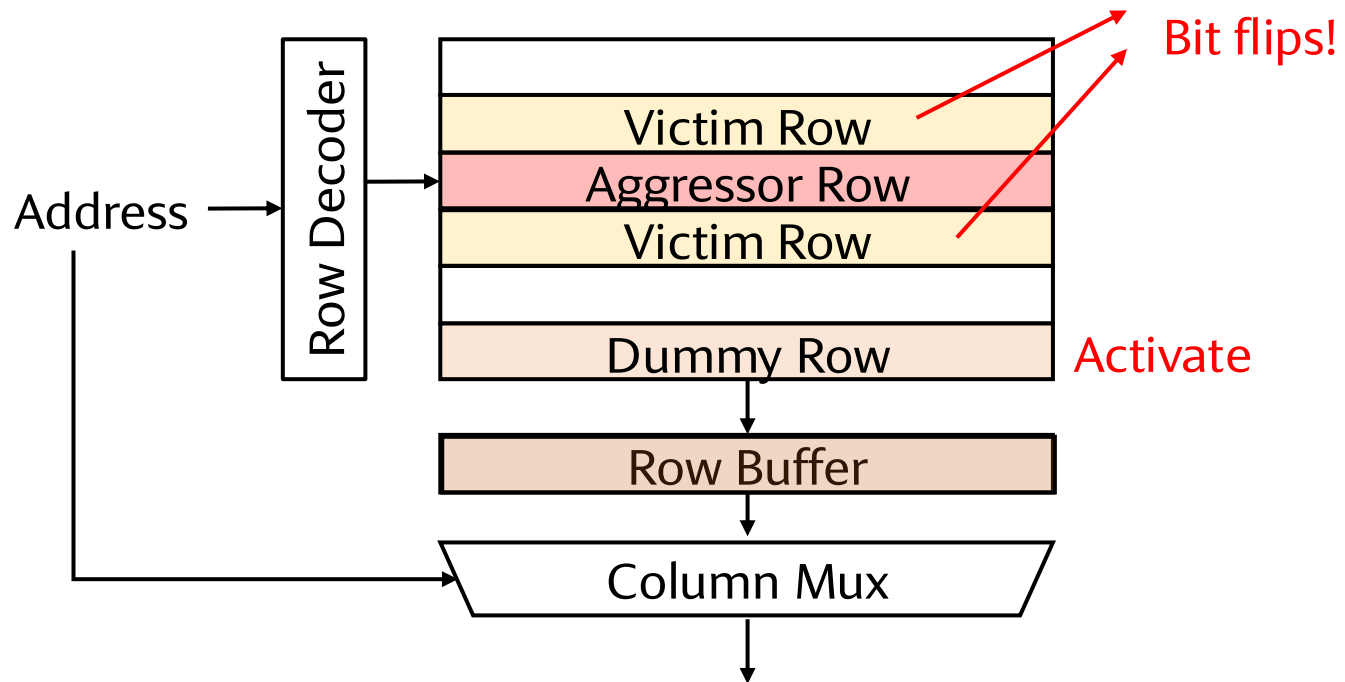
```
while (true) {  
    load hammer_row;  
  
    clflush hammer_row;  
    mfence;  
}
```

Hammering Sequence



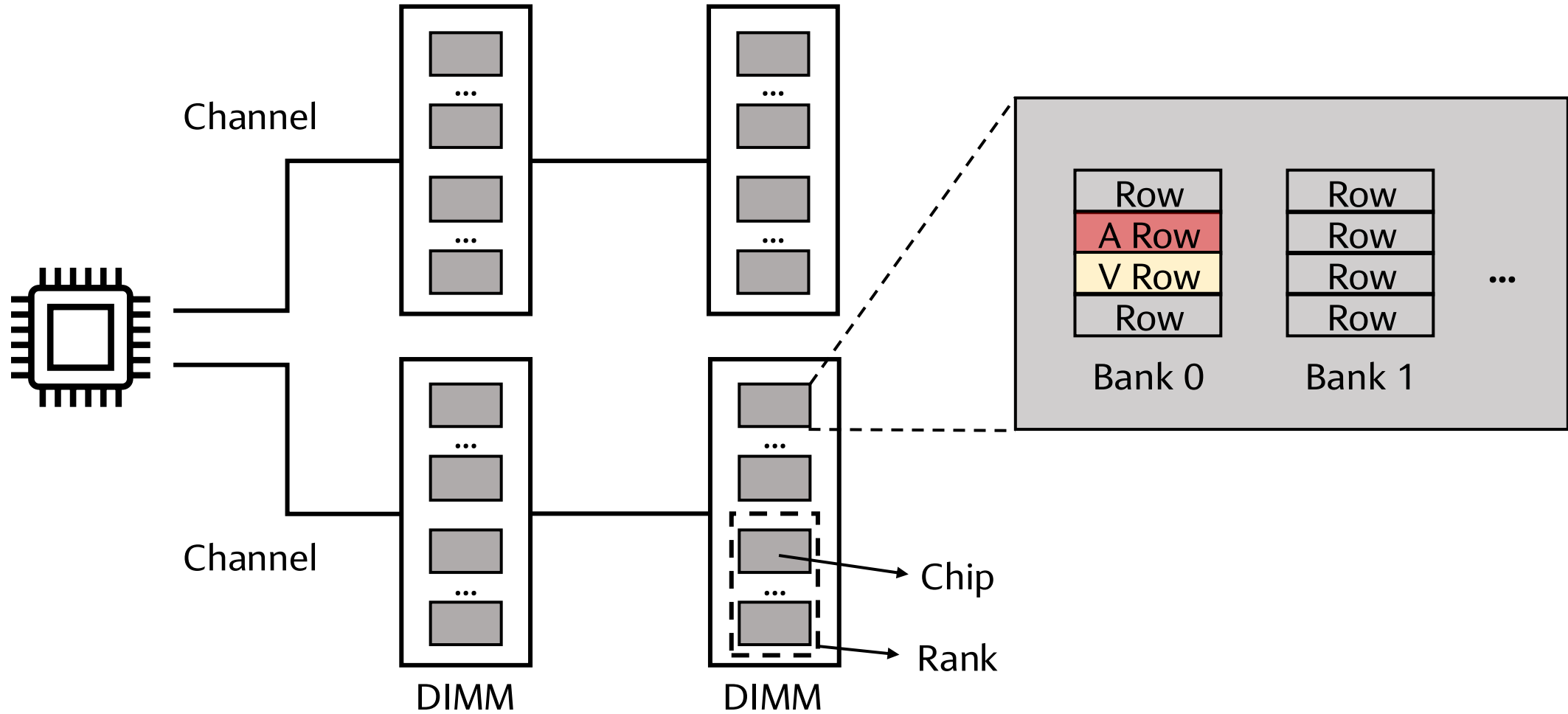
```
while (true) {  
    load hammer_row;  
    load dummy_row;  
  
    clflush hammer_row;  
    clflush dummy_row;  
    mfence;  
}
```

Hammering Sequence

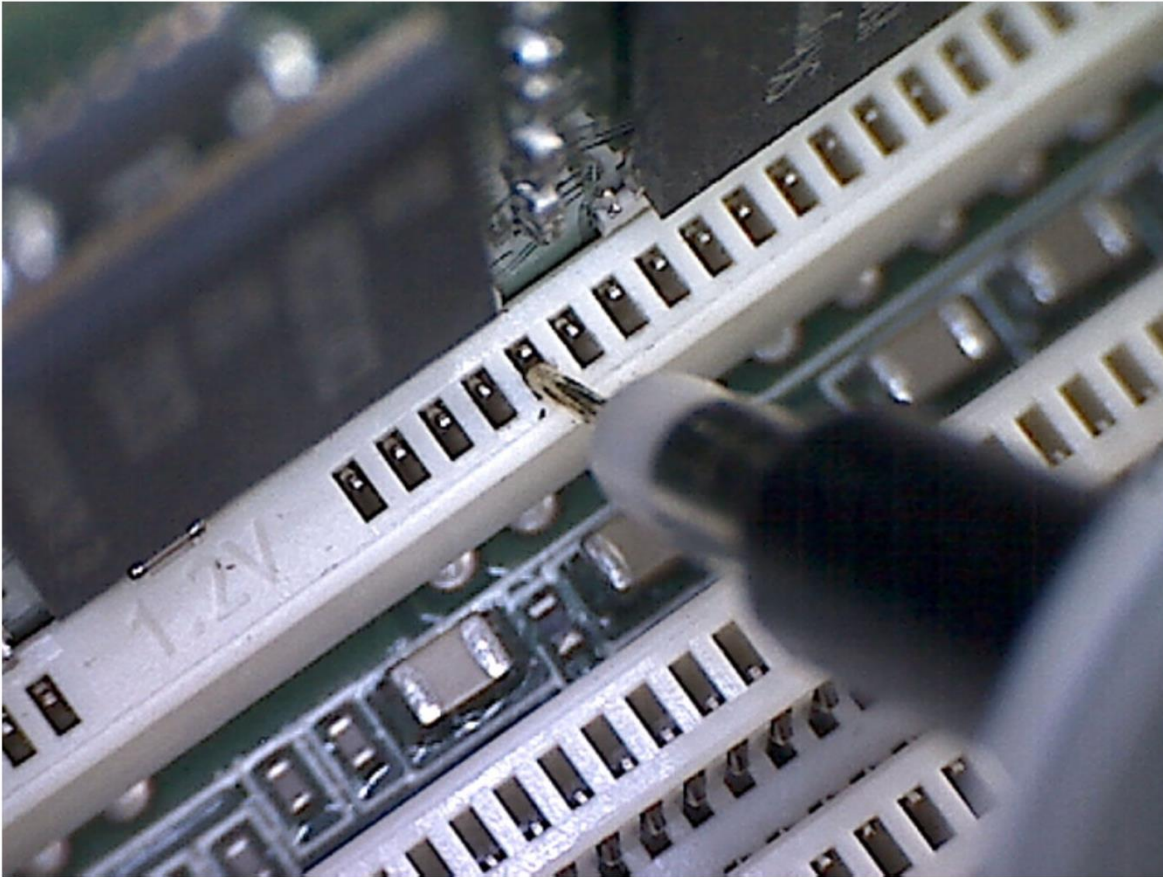


```
while (true) {  
    load hammer_row;  
    load dummy_row;  
  
    clflush hammer_row;  
    clflush dummy_row;  
    mfence;  
}
```

DRAM Mapping Function



Recover the Mapping Function



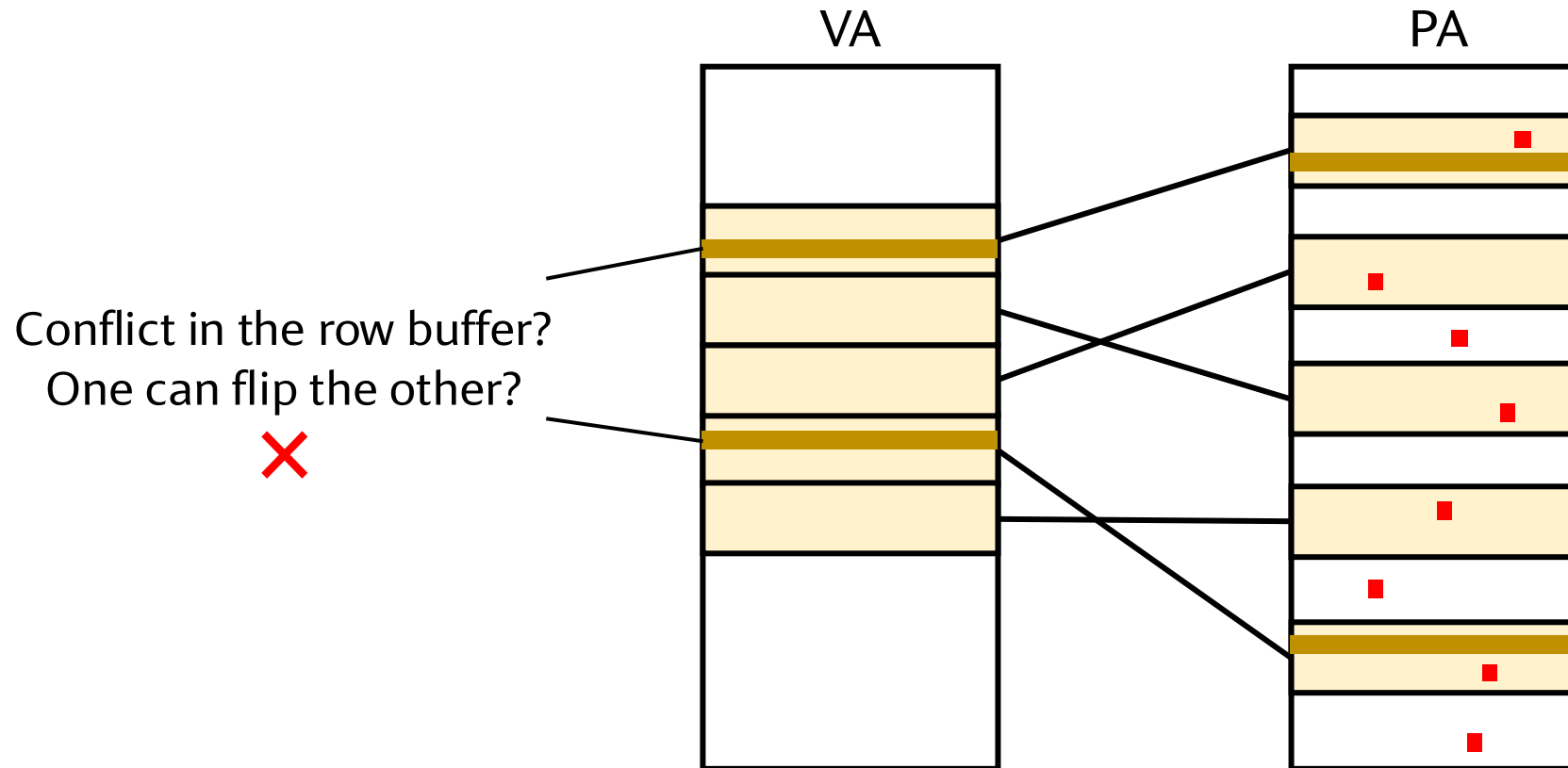
Method 1 (physical access): Repeated accesses to a known physical address, observe which rank and bank are used through **physical probing**

Method 2 (software-only): Detect addresses that are mapped to the same bank through row buffer conflicts
⇒ Timing difference

Source: Pessl et al, "DRAMA: Exploiting DRAM Addressing for Cross-CPU Attacks," USENIX '16

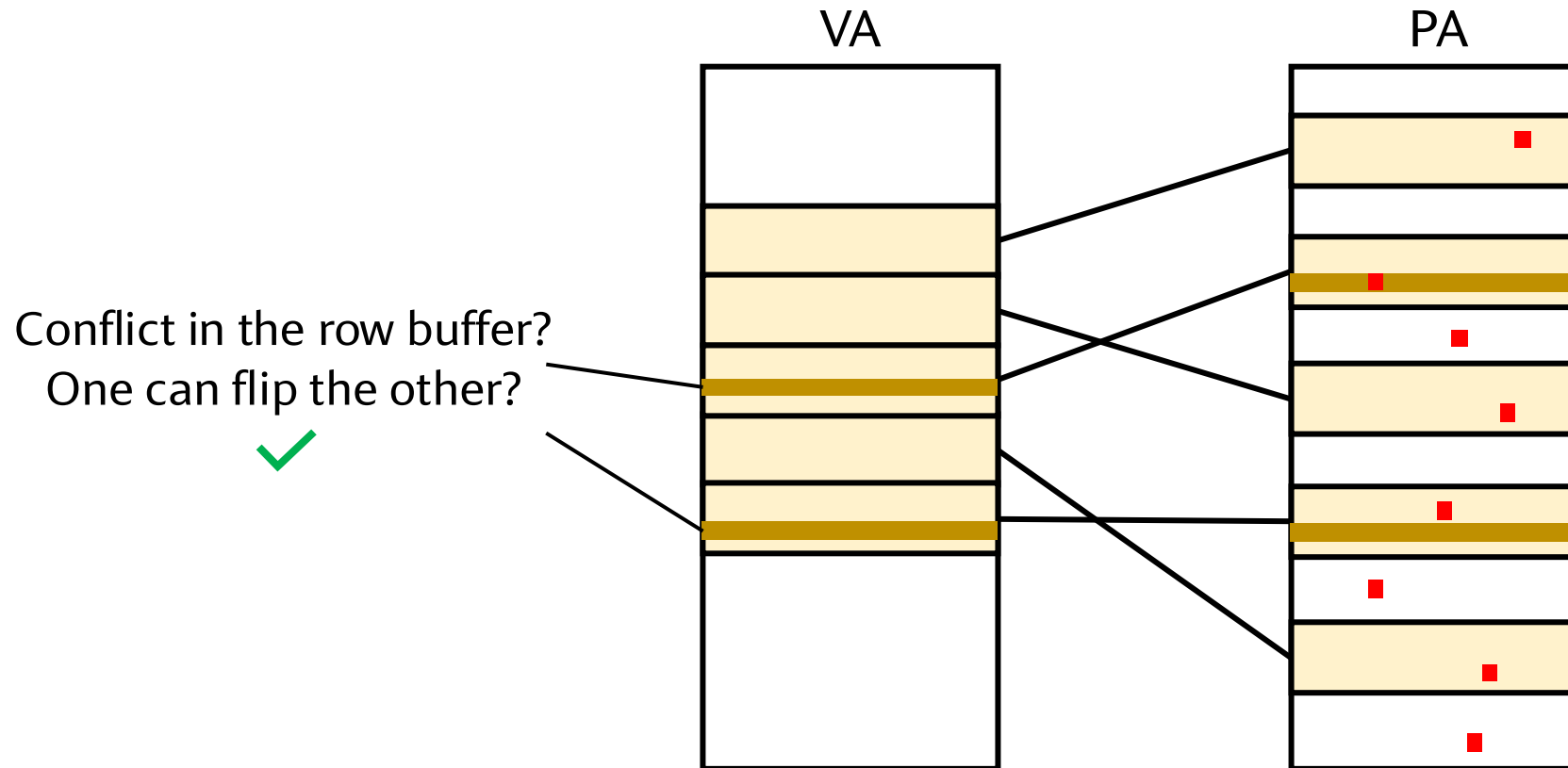
Finding Aggressor

Observation: Bit flips are repeatable



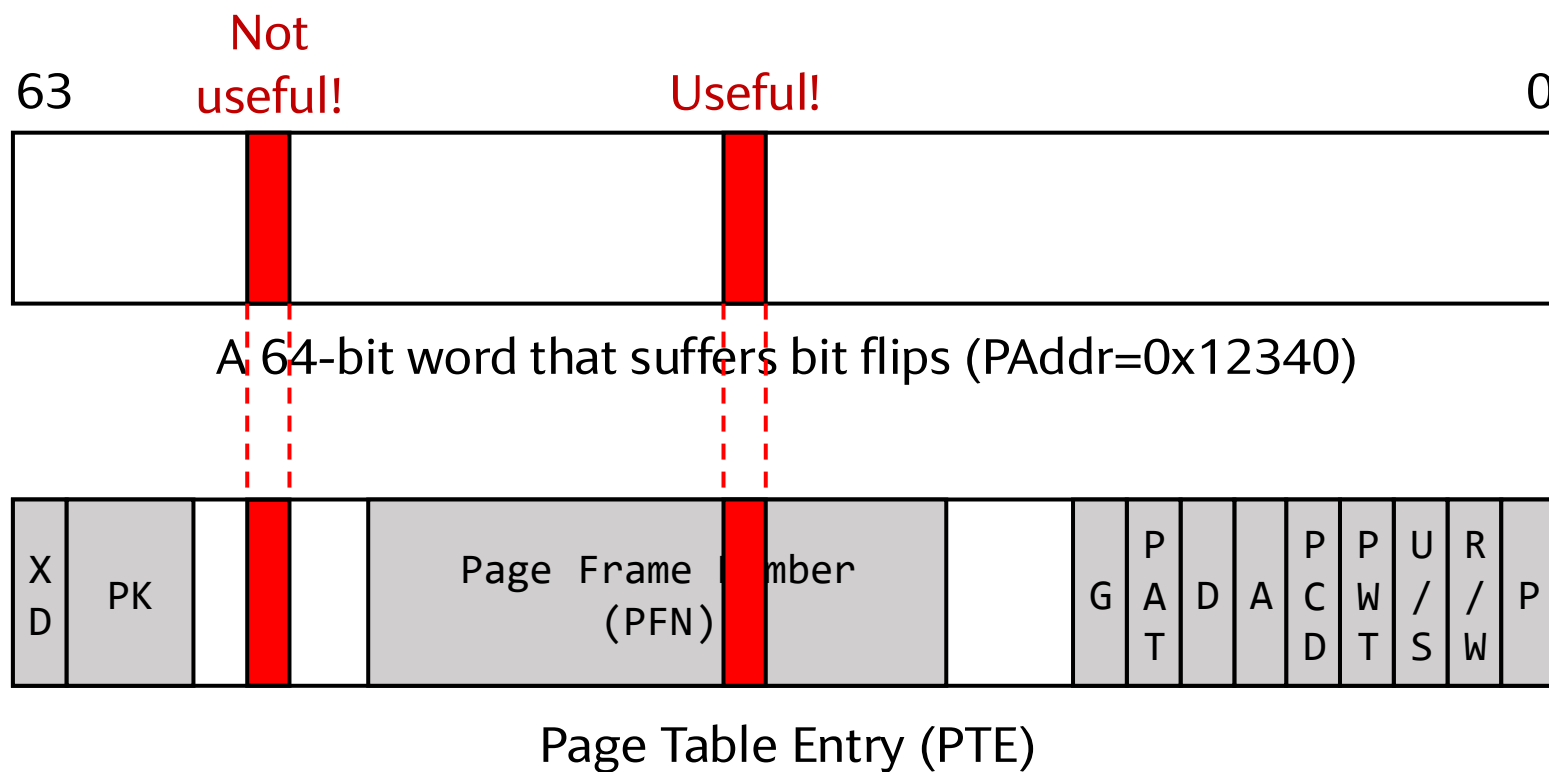
Finding Aggressor

Observation: Bit flips are repeatable



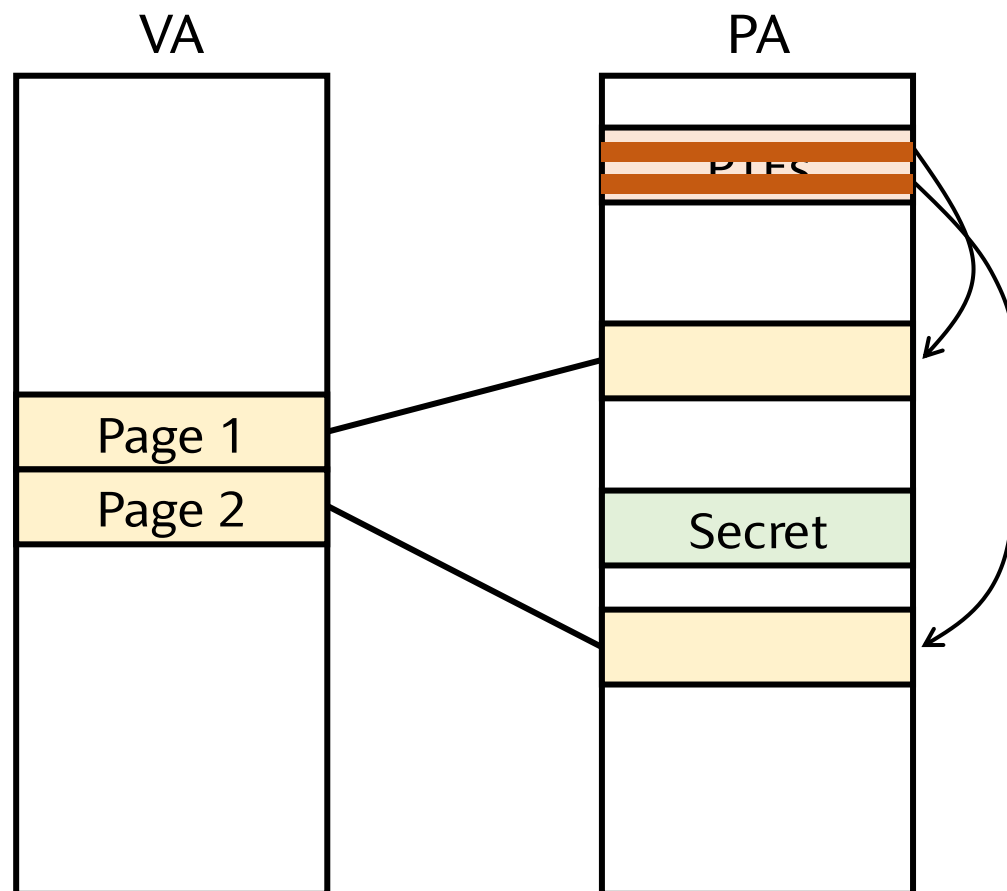
Is This Bit Flip Useful?

Goal: Corrupt the page-frame number of a page-table entry (PTE)



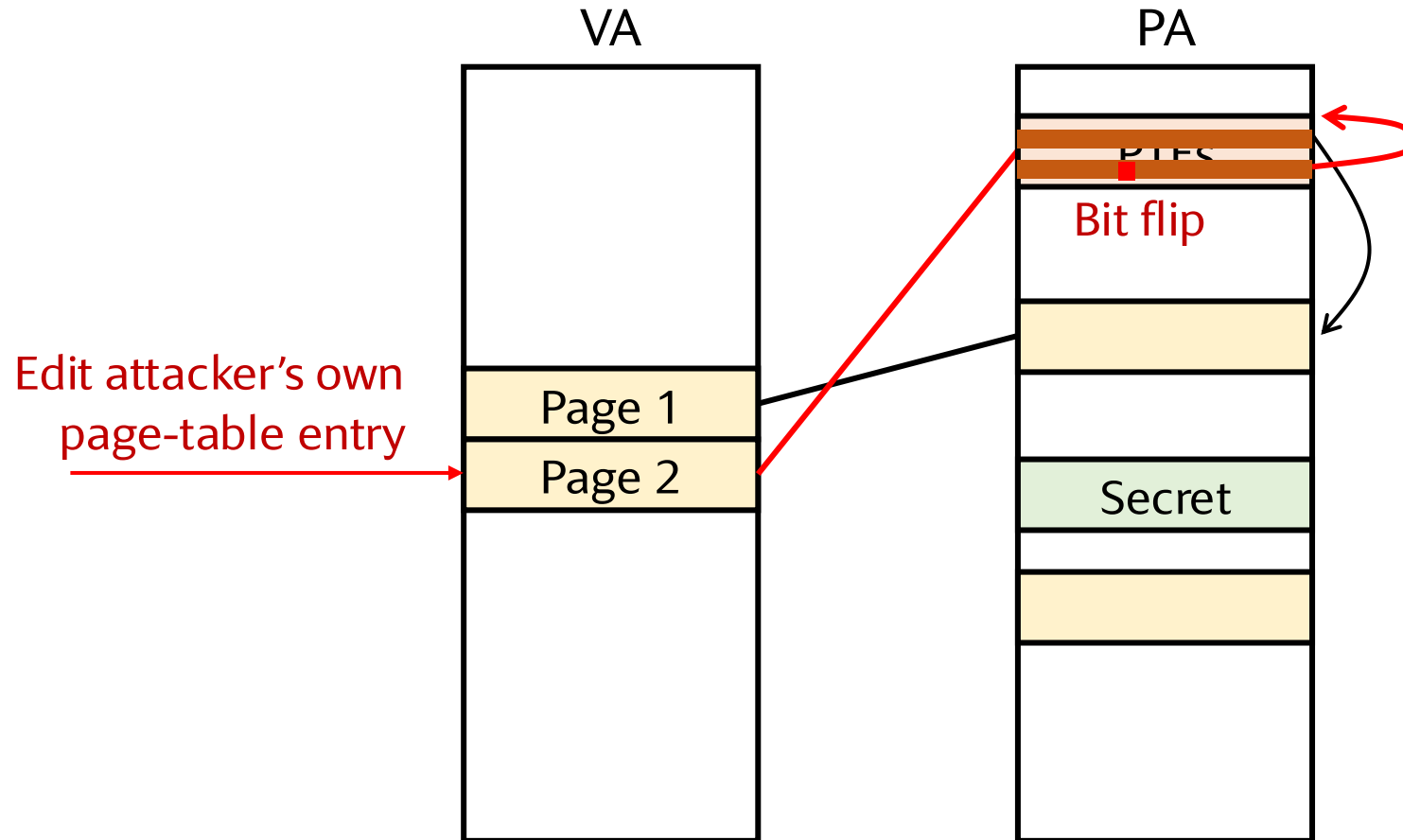
*<https://googleprojectzero.blogspot.com/2015/03/exploiting-dram-rowhammer-bug-to-gain.html>

Goal: Corrupt the Page Table Entry



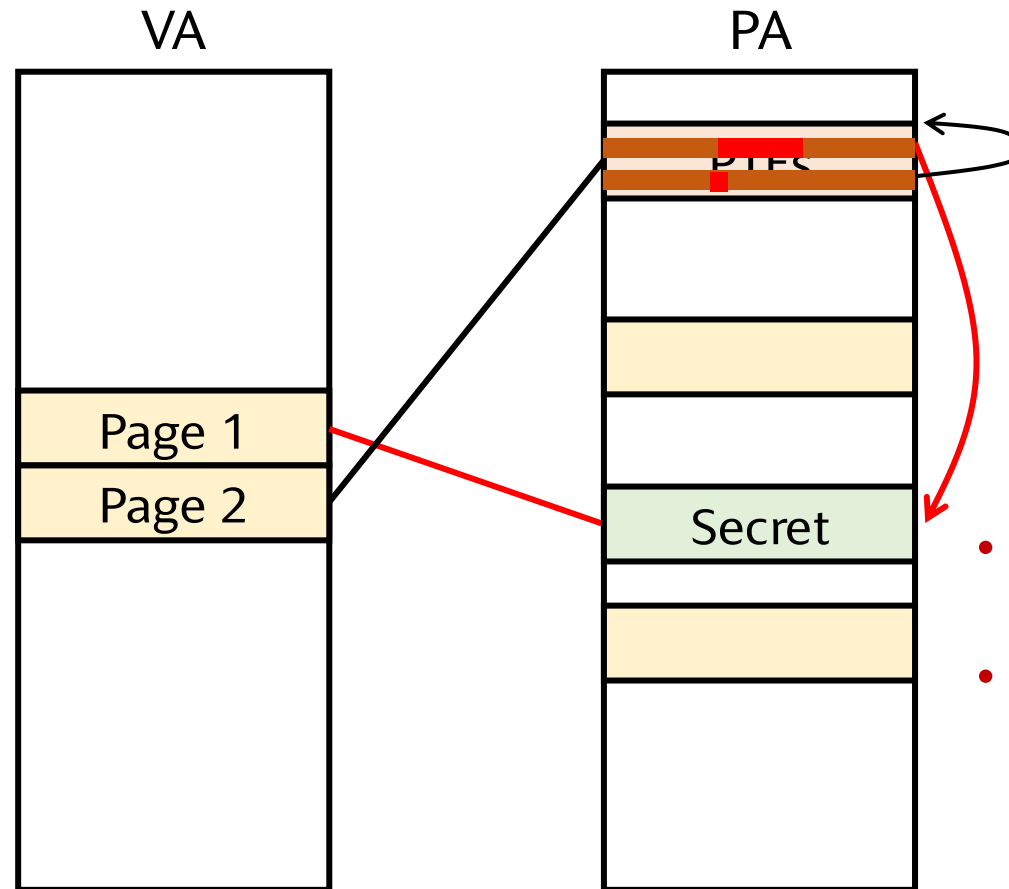
*<https://googleprojectzero.blogspot.com/2015/03/exploiting-dram-rowhammer-bug-to-gain.html>

Goal: Corrupt the Page Table Entry



*<https://googleprojectzero.blogspot.com/2015/03/exploiting-dram-rowhammer-bug-to-gain.html>

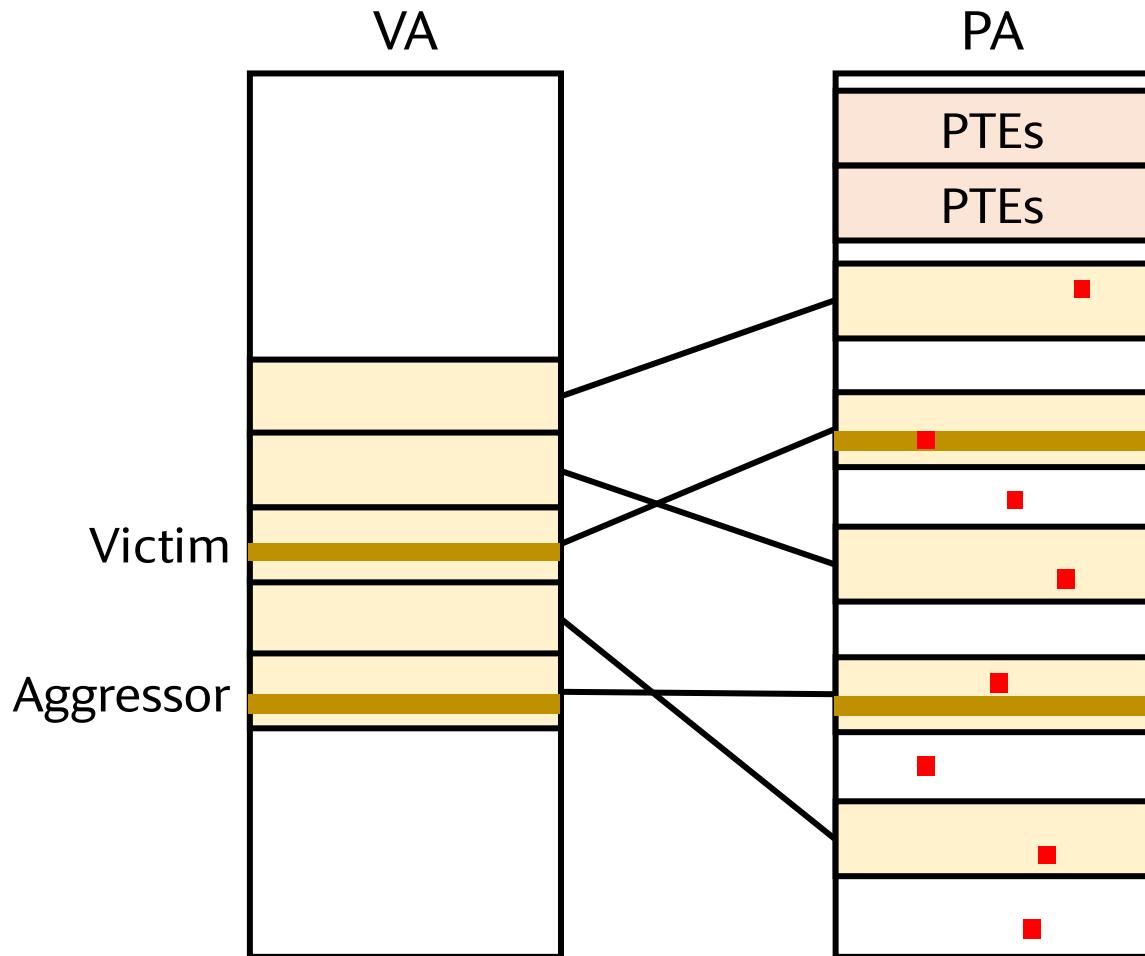
Goal: Corrupt the Page Table Entry



- How to trick the OS to put the PTE at a vulnerable address?
- How to ensure that the corrupted PTE points to a leaf page-table page?

*<https://googleprojectzero.blogspot.com/2015/03/exploiting-dram-rowhammer-bug-to-gain.html>

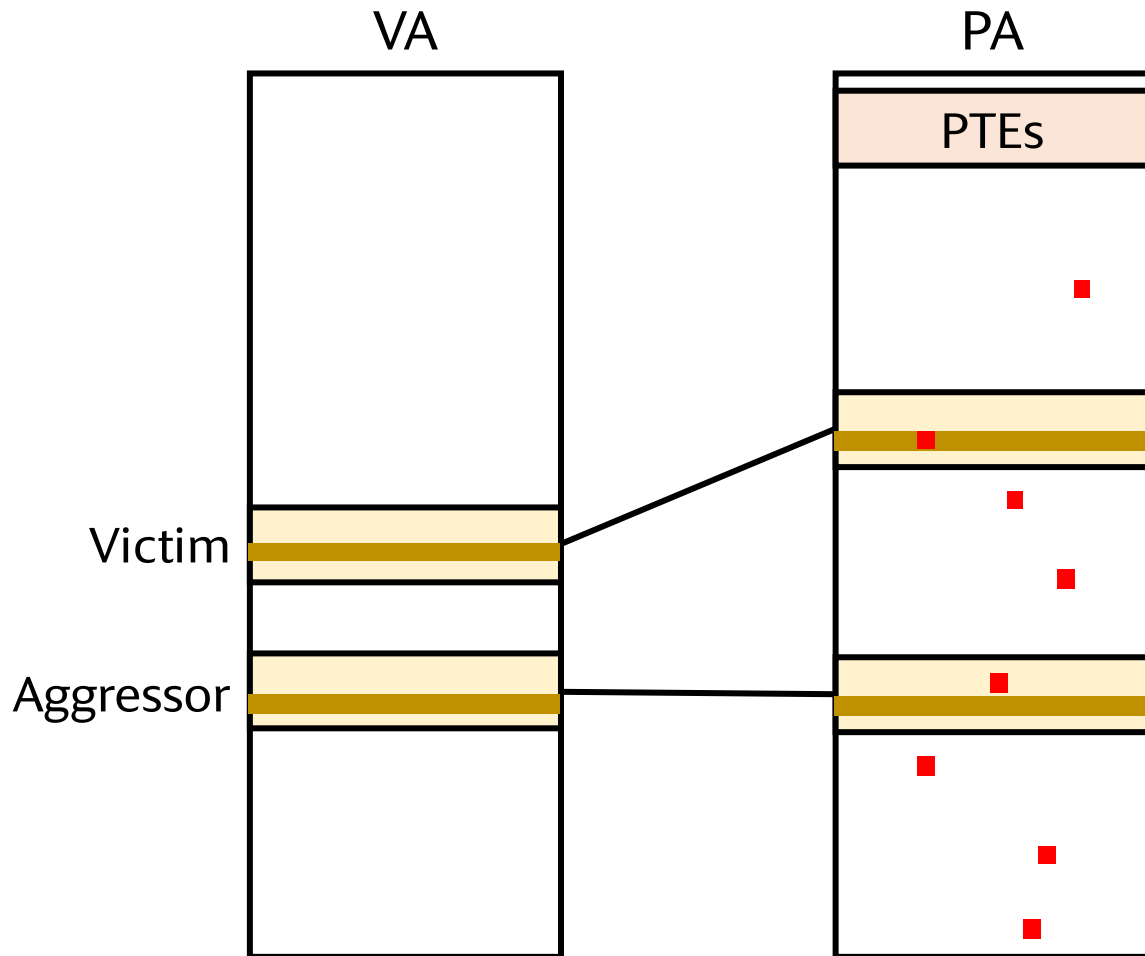
Memory Massaging



- Record the aggressor and victim addresses

*<https://googleprojectzero.blogspot.com/2015/03/exploiting-dram-rowhammer-bug-to-gain.html>

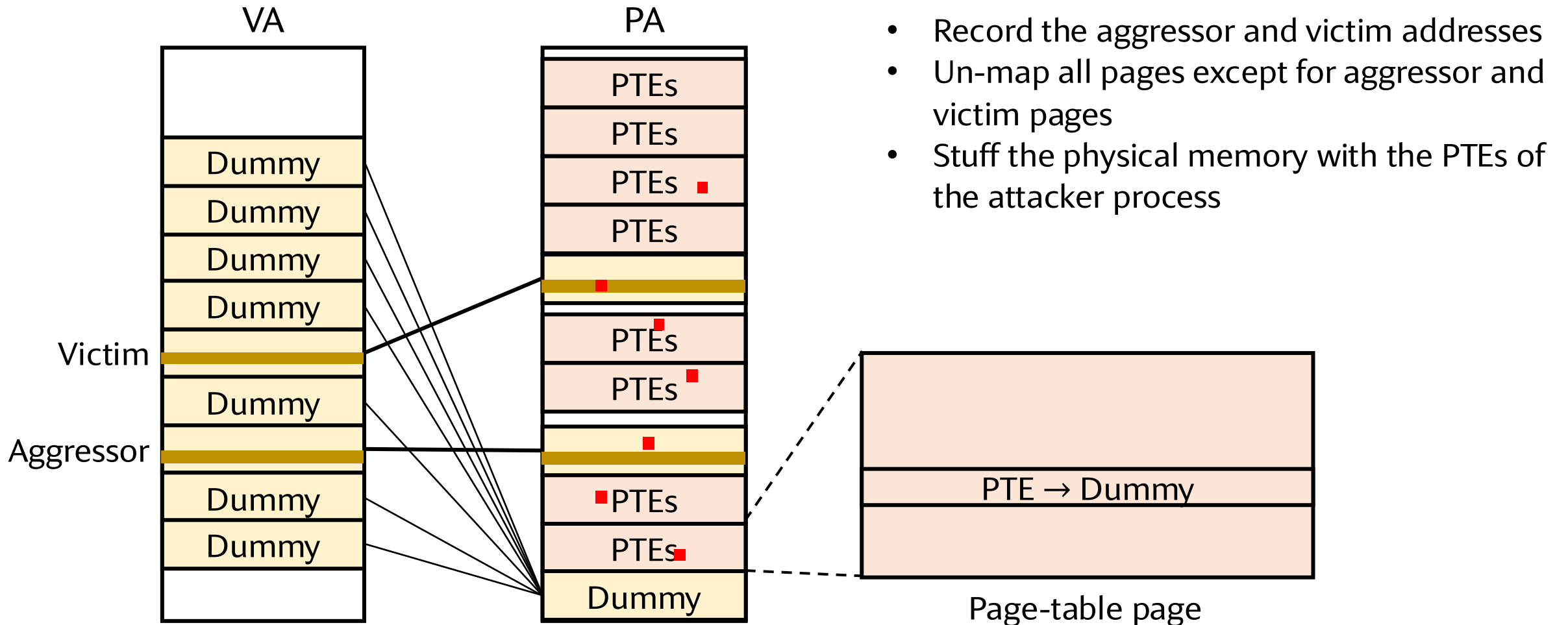
Memory Massaging



- Record the aggressor and victim addresses
- Un-map all pages except for aggressor and victim pages

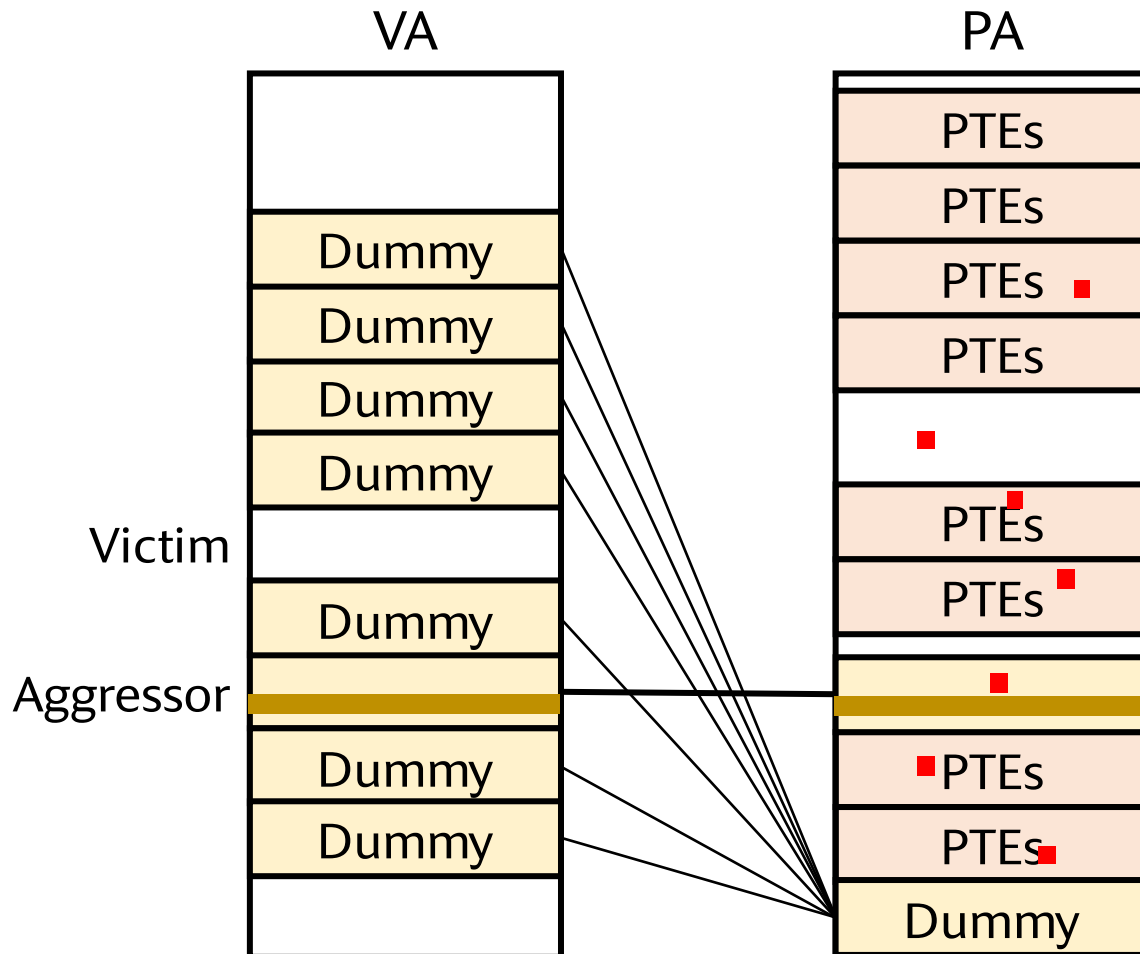
*<https://googleprojectzero.blogspot.com/2015/03/exploiting-dram-rowhammer-bug-to-gain.html>

Memory Massaging



*<https://googleprojectzero.blogspot.com/2015/03/exploiting-dram-rowhammer-bug-to-gain.html>

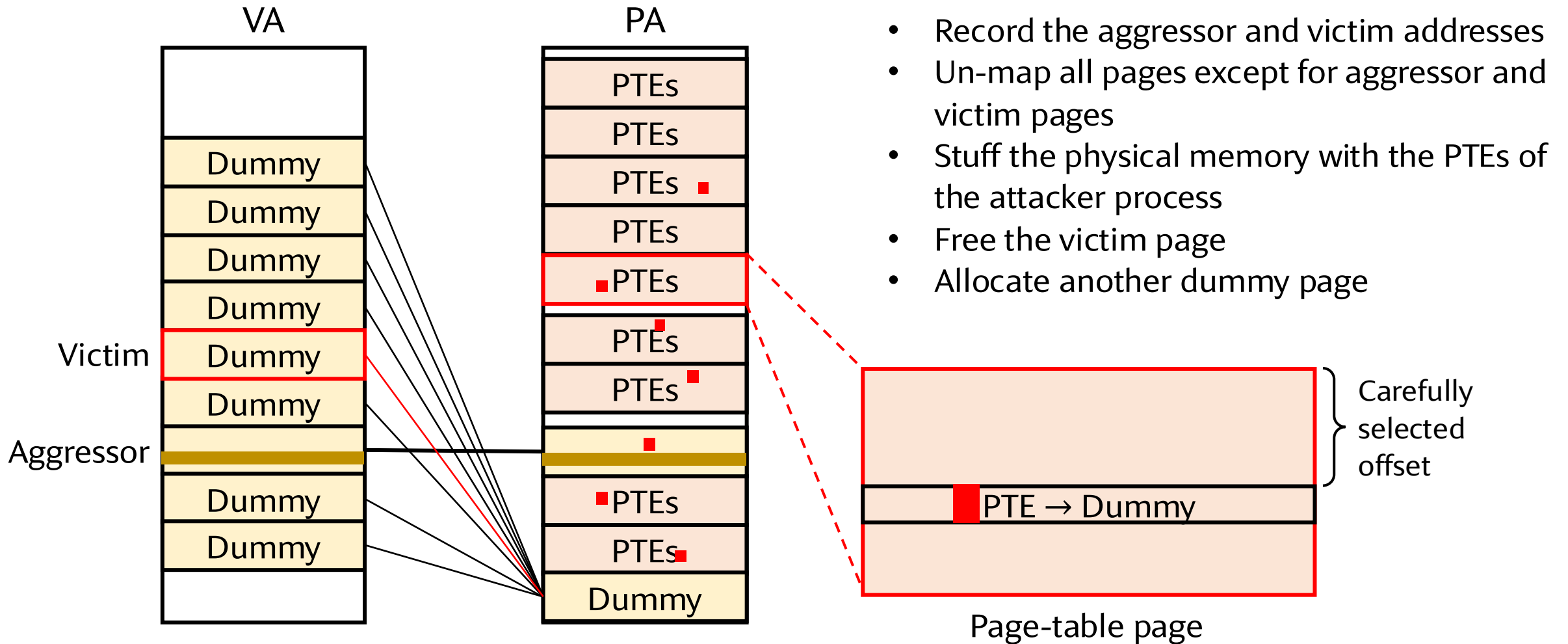
Memory Massaging



- Record the aggressor and victim addresses
- Un-map all pages except for aggressor and victim pages
- Stuff the physical memory with the PTEs of the attacker process
- Free the victim page

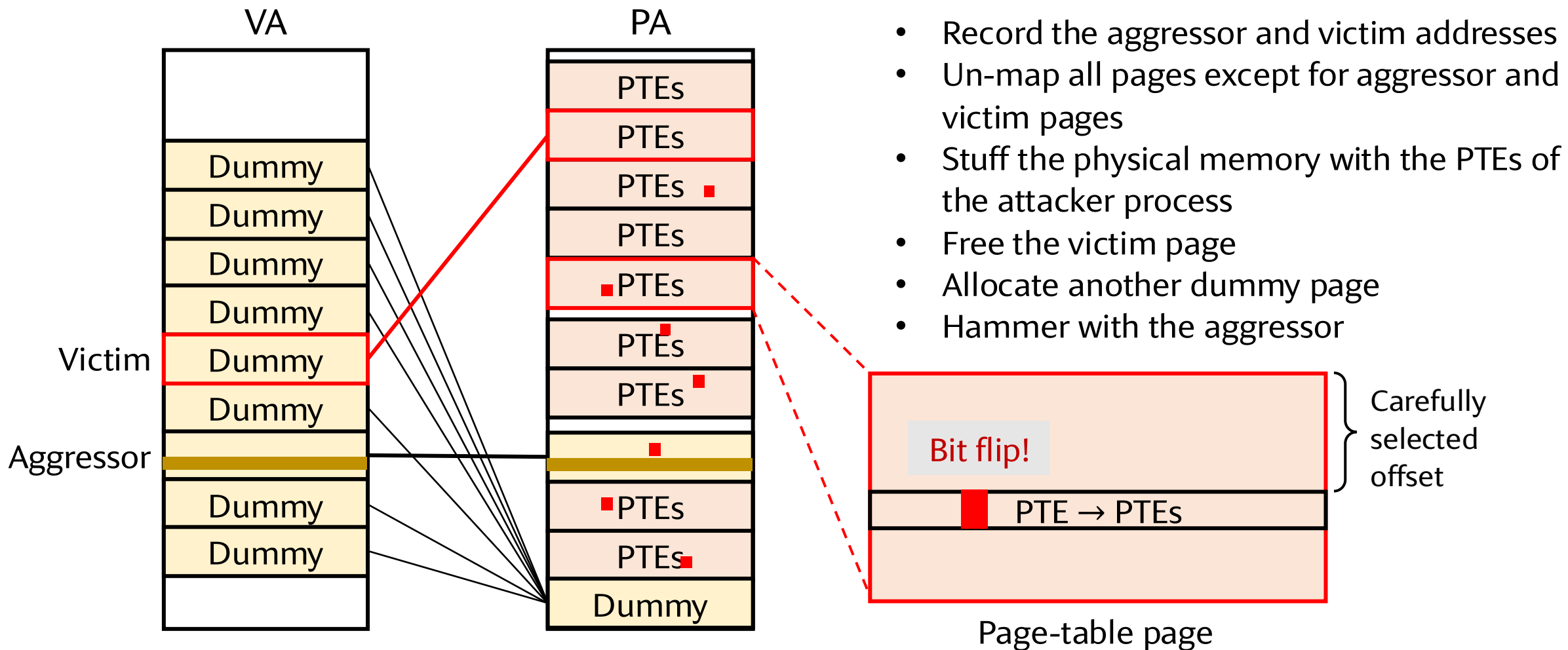
*<https://googleprojectzero.blogspot.com/2015/03/exploiting-dram-rowhammer-bug-to-gain.html>

Memory Massaging



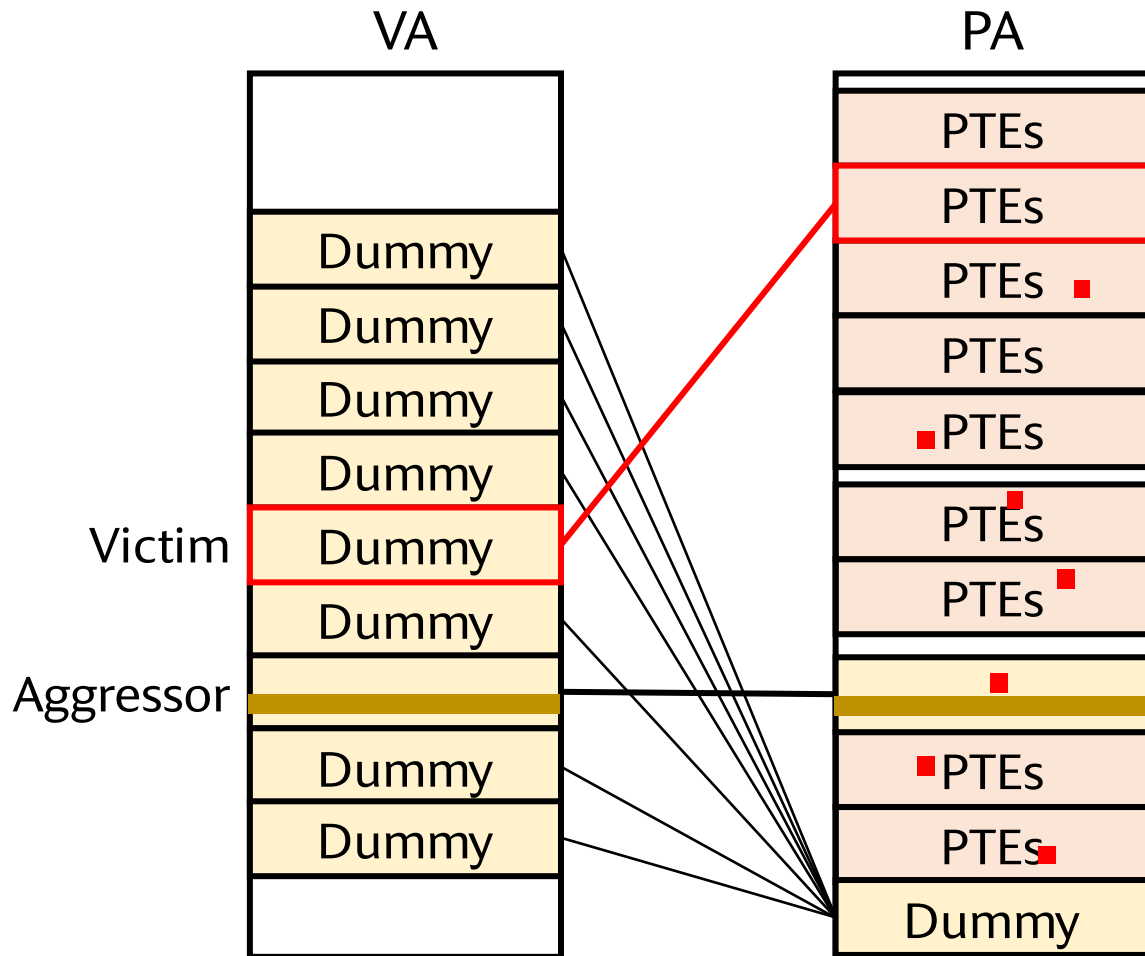
*<https://googleprojectzero.blogspot.com/2015/03/exploiting-dram-rowhammer-bug-to-gain.html>

Memory Massaging



- Record the aggressor and victim addresses
- Un-map all pages except for aggressor and victim pages
- Stuff the physical memory with the PTEs of the attacker process
- Free the victim page
- Allocate another dummy page
- Hammer with the aggressor

Memory Massaging



- Record the aggressor and victim addresses
- Un-map all pages except for aggressor and victim pages
- Stuff the physical memory with the PTEs of the attacker process
- Free the victim page
- Allocate another dummy page
- Hammer with the aggressor

⇒ The attacker process can write to its own page table entries

⇒ Access arbitrary physical address